

Алгоритми в JavaScript: вступ для новачка

Головна ідея: алгоритм -- це просто функція, яка перетворює вхідні дані на вихідні за певною логікою. Головне питання не "чи працює код", а "наскільки добре він масштабується" -- саме це вимірює Big O нотація. Розуміння масивів, циклів, рекурсії та базових структур даних -- це "будівельні блоки", з яких складається розв'язання будь-якої алгоритмічної задачі.

1. Що таке алгоритм і навіщо він потрібен

Алгоритм -- це послідовність кроків, яка перетворює вхідні дані (input) на результат (output). У коді це просто функція: щось приймає, щось повертає, а логіка всередині вирішує, як саме відбувається це перетворення.

"Добре сформульована задача -- наполовину розв'язана" (Charles Kettering). Перш ніж писати код, варто чітко визначити: що саме на вході, що очікується на виході, і які є граничні випадки (edge cases) -- порожній масив, від'ємні числа, дублікати тощо.

Навіщо вчити алгоритми, якщо є готові методи масивів?

- На співбесідах перевіряють не "правильну відповідь", а **хід міркувань** -- вміння розкласти задачу на кроки і обґрунтувати вибір підходу
- У реальних проєктах трапляються задачі, для яких немає готового методу -- і тоді потрібне розуміння, як побудувати рішення з нуля
- Розуміння того, "що під капотом" у звичних методах (`sort` , `find` , `includes`) допомагає передбачити, наскільки код буде швидким на великих обсягах даних

Як підступитись до будь-якої алгоритмічної задачі

- Чітко визнач вхід і вихід (найкраще -- у вигляді прикладів/тестів)
- Подумай про граничні випадки: порожній масив, одне значення, дуже великий вхід
- Прикинь, які змінні та цикли знадобляться -- чи є вбудований метод, що вже вирішує частину задачі
- Намалюй чи розпиши по кроках (діаграма чи псевдокод) -- це часто саме собою підказує рішення
- Подумай про масштабованість: як поведеться рішення, якщо вхідних даних стане в 1000 разів більше

2. Big O нотація: як виміряти "швидкість" алгоритму

Big O показує, як змінюється час (чи пам'ять), потрібні алгоритму, коли розмір вхідних даних зростає до нескінченності. Константи й дрібні доданки відкидаються -- залишається тільки та частина, яка домінує при дуже великих обсягах даних.

Позначення	Назва	Приклад
<code>O(1)</code>	Константний час	доступ до елемента масиву за індексом -- завжди однаково швидко
<code>O(log n)</code>	Логарифмічний	бінарний пошук у відсортованому масиві
<code>O(n)</code>	Лінійний	один прохід циклом по всьому масиву
<code>O(n log n)</code>	Лінійно-логарифмічний	ефективні сортування (merge sort, quick sort)
<code>O(n²)</code>	Квадратичний	два вкладені цикли (перевірка кожного з кожним)
<code>O(2ⁿ)</code>	Експоненційний	наївна рекурсивна функція Фібоначчі без кешування
<code>O(n!)</code>	Факторіальний	перебір усіх можливих перестановок (найгірший випадок)

Big O описує **найгірший випадок** (worst case) -- скільки максимально операцій знадобиться. Це важливо для прогнозування поведінки коду на великих обсягах даних, навіть якщо на малих прикладах різниця непомітна.

3. Масиви: базова структура даних

Масив -- найуживаніша структура даних у JS. Різні операції над масивом мають різну "вартість":

Операція	Складність	Чому
Доступ за індексом (<code>arr[i]</code>)	<code>O(1)</code>	індекс одразу вказує на комірку в пам'яті
Додавання в кінець (<code>push</code>)	<code>O(1)</code>	не треба зсувати інші елементи
Додавання/видалення на початку (<code>unshift</code> / <code>shift</code>)	<code>O(n)</code>	усі інші елементи треба перенумерувати (зсунути індекси)
Пошук без сортування (перебір)	<code>O(n)</code>	у гіршому випадку треба перевірити кожен елемент
Пошук у відсортованому масиві	<code>O(log n)</code>	можна ділити навпіл (бінарний пошук)

Практичне правило: віддавай перевагу `push` / `pop` (кінець масиву) над `unshift` / `shift` (початок масиву) там, де це можливо за логікою задачі -- перші два значно дешевші.

4. Ітерації (цикли)

Конструкція	Коли використовувати
<code>for</code>	найгнучкіший -- повний контроль над індексом, напрямком, кроком
<code>while</code> / <code>do...while</code>	коли кількість ітерацій наперед невідома, є лише умова зупинки
<code>for...in</code>	перебір ключів об'єкта
<code>for...of</code>	перебір значень будь-якої ітерабельної структури (масив, рядок, Map, Set)
Методи з колбеком (<code>map</code> , <code>filter</code> , <code>reduce</code> , <code>find</code> та інші)	коли треба трансформувати/відфільтрувати/знайти -- без ручного керування лічильником

Методи, що самі проходять по всьому масиву без явного колбека: `indexOf` , `lastIndexOf` , `includes` , `fill` , `join` . Методи, що приймають колбек-функцію: `find` , `findIndex` , `filter` , `forEach` , `map` , `some` , `every` , `reduce` .

5. Рекурсія

Рекурсивна функція викликає сама себе, поки не досягне **базового випадку** (base case) -- умови, за якої вона більше не викликає себе, а просто повертає значення.

```
// Ітеративний варіант факторіала
const factorial = (number) => {
  let product = 1;
  for (let i = 2; i <= number; i++) product *= i;
  return product;
};

// Рекурсивний варіант -- той самий результат в один рядок
const factorial = (number) => number < 2 ? 1 : number * factorial(number - 1);
```

Кожен виклик функції додає новий "кадр" у стек викликів (call stack). Якщо базовий випадок ніколи не настає (або рекурсія надто глибока), стек переповнюється -- виникає помилка **stack overflow**. Будь-яку рекурсивну функцію теоретично можна переписати як ітеративну (цикл), і навпаки -- вибір залежить від того, що зрозуміліше для конкретної задачі.

6. Патерни розв'язання задач (шукай ці підходи в задачах)

Патерн	Суть	Приклад задачі
Two pointers (два вказівники)	два індекси рухаються назустріч або в одному напрямку по масиву/рядку	перевірка паліндрому, розворот масиву без нового масиву
Sliding window (ковзне вікно)	тримати "вікно" фіксованого чи змінного розміру і зсувати його по масиву	найдовший підрядок без повторів, сума підмасиву заданої довжини
Frequency counter (частотна таблиця)	об'єкт/Map, що рахує входження кожного елемента, замість вкладених циклів	анаграми, пошук символу, що зустрічається найчастіше

Frequency counter (хеш-таблиця/об'єкт) майже завжди перетворює наївне рішення з $O(n^2)$ (вкладені цикли -- порівняння кожного з кожним) на $O(n)$, ціною додаткової пам'яті на саму таблицю. Це один із найважливіших "перших трюків" для оптимізації.

```
// Замість вкладеного циклу для пошуку дублікатів (O(n²))
// -- використай Set для пошуку за O(1) на кожній перевірці:
function hasDuplicate(arr) {
  const seen = new Set();
  for (const item of arr) {
    if (seen.has(item)) return true; // O(1) перевірка замість arr.includes() -- O(n)
    seen.add(item);
  }
  return false;
} // Загалом O(n) замість O(n²)
```

7. Основні структури даних (крім масивів)

Структура	Принцип	Типове застосування
Стек (Stack)	LIFO -- останній додано, перший вилучено	історія дій (undo), стек викликів функцій
Черга (Queue)	FIFO -- перший додано, перший вилучено	черга задач на виконання, обробка запитів по порядку
Однозв'язний список (Linked List)	кожен елемент (вузол) містить дані й посилання на наступний	ефективна вставка/видалення без зсуву всіх елементів, як у масиві
Дерево (Tree)	ієрархічна структура: корінь і дочірні вузли	DOM-дерево сторінки, файлова система, ієрархія категорій

Дерево можна обходити двома способами: **breadth-first** (по рівнях, за допомогою черги) або **depth-first** (углиб, за допомогою стека чи рекурсії).

8. Алгоритми сортування (огляд)

Алгоритм	Складність (час)	Суть
Bubble sort	$O(n^2)$	порівнює сусідні елементи, "спливають" найбільші -- найпростіший, але найповільніший
Insertion sort	$O(n^2)$	вставляє кожен новий елемент у вже відсортовану частину
Selection sort	$O(n^2)$	на кожному кроці знаходить мінімум і ставить на початок
Quick sort	$O(n \log n)$ у середньому	обирає опорний елемент (pivot), ділить на менші/більші -- найшвидший на практиці
Merge sort	$O(n \log n)$	рекурсивно ділить навпіл, потім зливає відсортовані половини
Counting sort	$O(n)$	працює тільки коли відомий діапазон значень -- рахує входження кожного значення

Вбудований метод масиву `array.sort()` у сучасних браузерах працює за $O(n \log n)$ -- використовуй його за замовчуванням, якщо немає особливих вимог до стабільності чи діапазону значень.

9. Пошук: лінійний vs бінарний

Спосіб	Умова	Складність
Лінійний пошук	масив у будь-якому порядку	$O(n)$ -- у гіршому випадку перевіряється кожен елемент
Бінарний пошук	масив має бути відсортований	$O(\log n)$ -- щоразу відкидає половину, де значення точно немає

Якщо потрібно шукати в одному й тому самому масиві багато разів -- варто один раз відсортувати його ($O(n \log n)$), а потім користуватись швидким бінарним пошуком ($O(\log n)$) на кожен запит.

10. Швидкий довідник

Поняття	Суть
Алгоритм	послідовність кроків, що перетворює вхідні дані на результат
Big O	нотація, що описує, як зростає час/пам'ять алгоритму зі зростанням вхідних даних
$O(1)$ / $O(\log n)$ / $O(n)$	константний / логарифмічний / лінійний час -- від найкращого до помірного
$O(n \log n)$ / $O(n^2)$ / $O(2^n)$	лінійно-логарифмічний / квадратичний / експоненційний -- зростаюча "вартість"
Space complexity	скільки додаткової пам'яті потребує алгоритм (аналог Big O, але для пам'яті)
Base case	умова зупинки рекурсії -- без неї виникає stack overflow
Two pointers	два індекси рухаються по структурі одночасно
Sliding window	вікно фіксованого/змінного розміру зсувається по масиву/рядку
Frequency counter	хеш-таблиця (об'єкт/Map/Set) замість вкладених циклів -- $O(n)$ замість $O(n^2)$
Stack / Queue	LIFO / FIFO структури даних
Linked List	елементи-вузли з посиланням на наступний, ефективна вставка/видалення
Tree / BST	ієрархічна структура; Binary Search Tree дозволяє пошук за $O(\log n)$
Bubble / Insertion / Selection sort	прості, але $O(n^2)$ сортування
Quick / Merge sort	ефективні $O(n \log n)$ сортування
Лінійний / бінарний пошук	$O(n)$ без сортування / $O(\log n)$ у відсортованому масиві